Sprawozdanie – Laboratorium 2: Architektura warstwowa, walidacja i obsługa błędów

Przedmiot: Zaawansowane Technologie Programowania Aplikacji Internetowych

Temat: Room Booking API (architektura i reguły biznesowe)

Student: Kacper Gumulak – 151872

GitHub: github.com/zephir-x/pk-ztpai

1. Krótki opis rozwiązania

Zadanie polegało na przebudowie aplikacji Room Booking API z pierwszego laboratorium na architekturę warstwową przy użyciu środowiska Node.js, frameworka ExpressJS 5 oraz biblioteki walidacyjnej Zod. Zgodnie z założeniami, aplikacja nadal nie korzysta z zewnętrznej bazy danych, natomiast cały stan w pamięci operacyjnej został odizolowany i zamknięty wyłącznie w warstwie repozytoriów (`*.repository.memory.js`), których metody są asynchroniczne (`async / Promise`), a identyfikatory nowych rezerwacji są generowane przez wewnętrzny licznik.

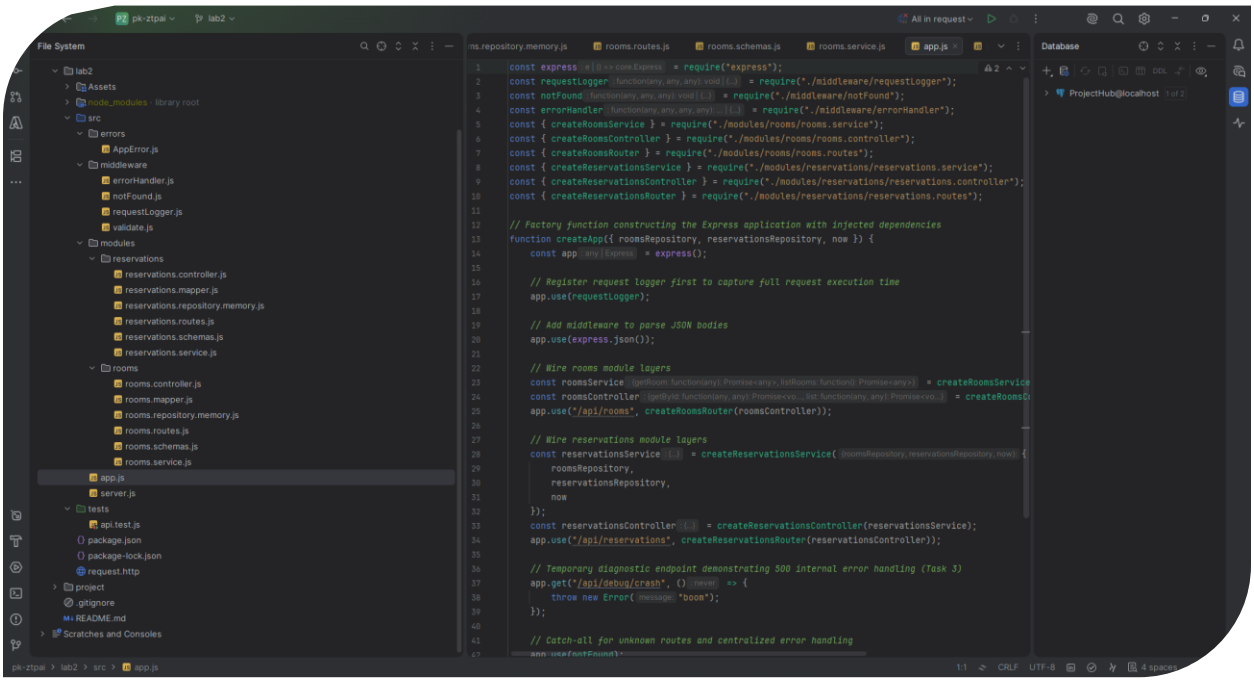
Model rezerwacji został rozszerzony o przedział czasowy (`startTime, endTime`), znacznik utworzenia (`createdAt`) oraz anulowania (`cancelledAt`). Wprowadzono podział odpowiedzialności na warstwy: `routes → controllers → services → repositories`, wstrzykiwanie zależności (Dependency Injection) dla repozytoriów oraz aktualnego czasu (now), deklaratywną walidację danych wejściowych przy pomocy schematów Zod, centralną obsługę błędów (`AppError, errorHandler`) wraz z logowaniem czasu żądań i nagłówkiem `X-Request-Id`, a także separację modelu wewnętrznego od publicznego kontraktu API poprzez obiekty DTO (mappery).

2. Realizacja zadań

Zadanie 1: Architektura warstwowa

Aplikacja została podzielona na moduły funkcjonalne (rooms oraz reservations), warstwy pośrednie (middleware) oraz klasy błędów (errors). Konfiguracja aplikacji Express została wydzielona do funkcji fabrykującej createApp w pliku src/app.js, natomiast plik src/server.js pełni rolę composition root – tworzy instancje repozytoriów i uruchamia nasłuchiwanie serwera HTTP.

Drzewo katalogów projektu (lab2):



Zrzut ekranu z Ridera pokazujący pełne drzewo katalogów projektu.

Fragment kodu – pełny przepływ (route → controller → service → repository) na przykładzie modułu rooms:

```
// 1. Route (src/modules/rooms/rooms.routes.js)
function createRoomsRouter(controller) {
    const router = Router();

    router.get("/", controller.list);
    router.get("/:id", validate({ params: idParams }), controller.getById);

    return router;
}
```

```
// 2. Controller (src/modules/rooms/rooms.controller.js)
function createRoomsController(roomsService) {
    return {
        async list(req, res) {
            const rooms = await roomsService.listRooms();
            res.json(rooms.map(toRoomResponse));
        },

        async getById(req, res) {
            const room = await roomsService.getRoom(req.validated.params.id);
            res.json(toRoomResponse(room));
        }
    };
}

// 3. Service (src/modules/rooms/rooms.service.js)
function createRoomsService({ roomsRepository }) {
    async function listRooms() {
        return roomsRepository.list();
    }

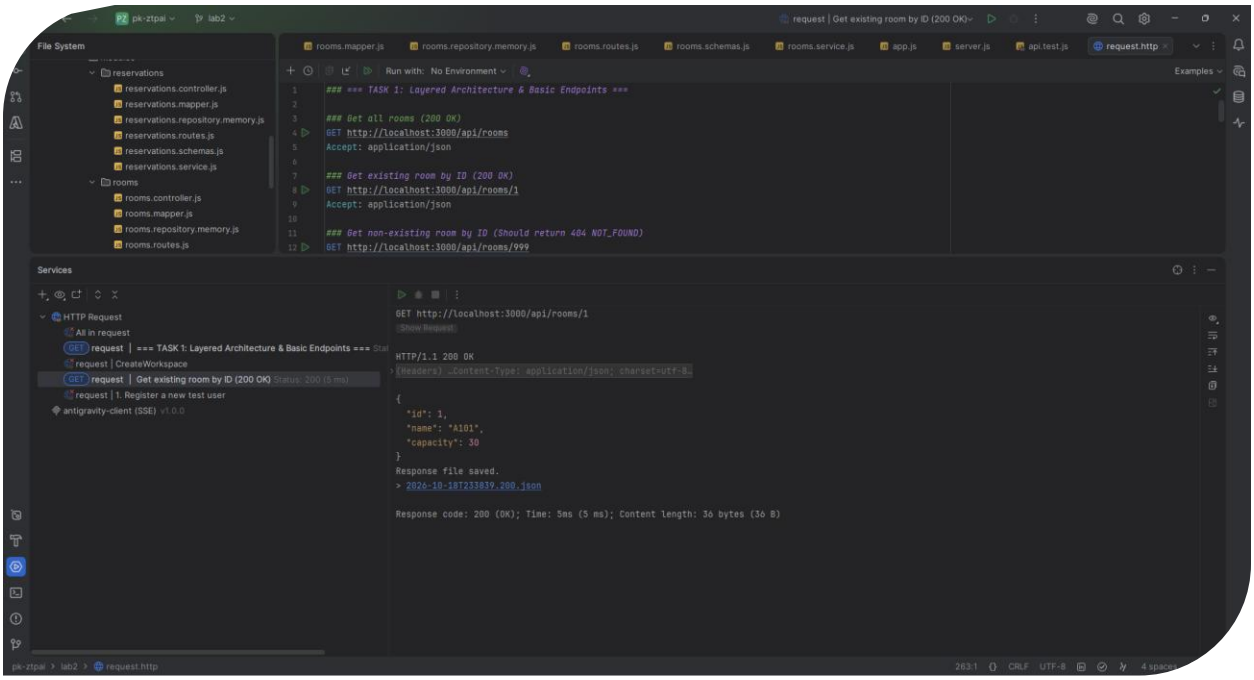
    async function getRoom(id) {
        const room = await roomsRepository.findById(id);
        if (!room) {
            throw new NotFoundError("Room");
        }
        return room;
    }

    return { listRooms, getRoom };
}

// 4. Repository (src/modules/rooms/rooms.repository.memory.js)
function createInMemoryRoomsRepository(seed = DEFAULT_ROOMS) {
    const state = {
        rooms: structuredClone(seed)
    };

    return {
        async list() {
            return state.rooms;
        },

        async findById(id) {
            return state.rooms.find(r => r.id === id) ?? null;
        }
    };
}
```



Zrzut ekranu z Ridera pokazujący poprawny request GET http://localhost:3000/api/rooms/1 (kod 200 OK wraz z nagłówkiem X-Request-Id).

Zadanie 2: Validation z Zod

Ręczne instrukcje warunkowe zostały zastąpione deklaratywnymi schematami biblioteki Zod oraz jednym uniwersalnym middlewarem `validate({ params, query, body })`. Middleware parsuje dane za pomocą safeParse, zbiera wszystkie błędy walidacji naraz i przekazuje oczyszczone dane (np. po `.trim()` i konwersji typów `z.coerce.number()`) w obiekcie `req.validated`. Użycie metody `.strict()` blokuje przesyłanie nieznanych pól (ochrona przed mass assignment).

Fragment kodu (uniwersalny middleware validate oraz schematy rezerwacji):

```
// src/middleware/validate.js
const { ValidationError } = require("../errors/AppError");

function validate(schemas) {
    return (req, res, next) => {
        const validated = {};
        const details = [];

        for (const part of ["params", "query", "body"]) {
            if (!schemas[part]) continue;

            const result = schemas[part].safeParse(req[part] ?? {});
            if (!result.success) {
                for (const issue of result.error.issues) {
```

```
                    details.push({
                        location: part,
                        field: issue.path.join("."),
                        message: issue.message
                    });
                }
            } else {
                validated[part] = result.data;
            }
        }

        if (details.length > 0) {
            return next(new ValidationError(details));
        }

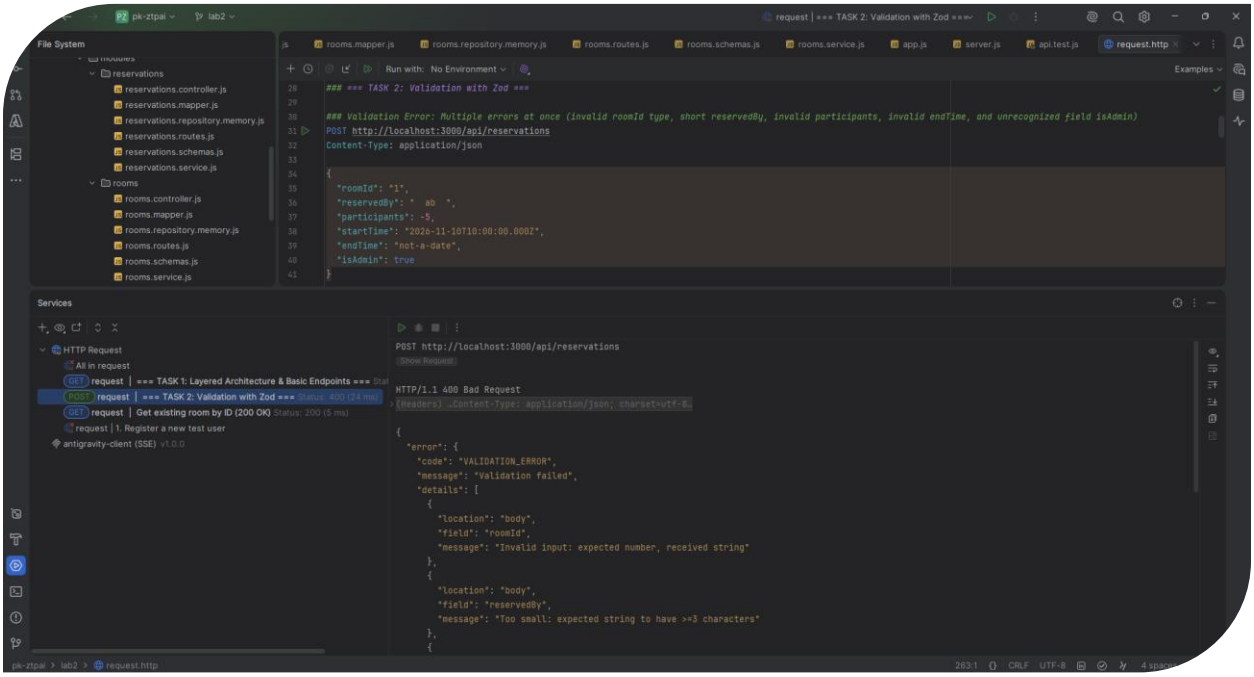
        req.validated = validated;
        next();
    };
}

// src/modules/reservations/reservations.schemas.js
const { z } = require("zod");

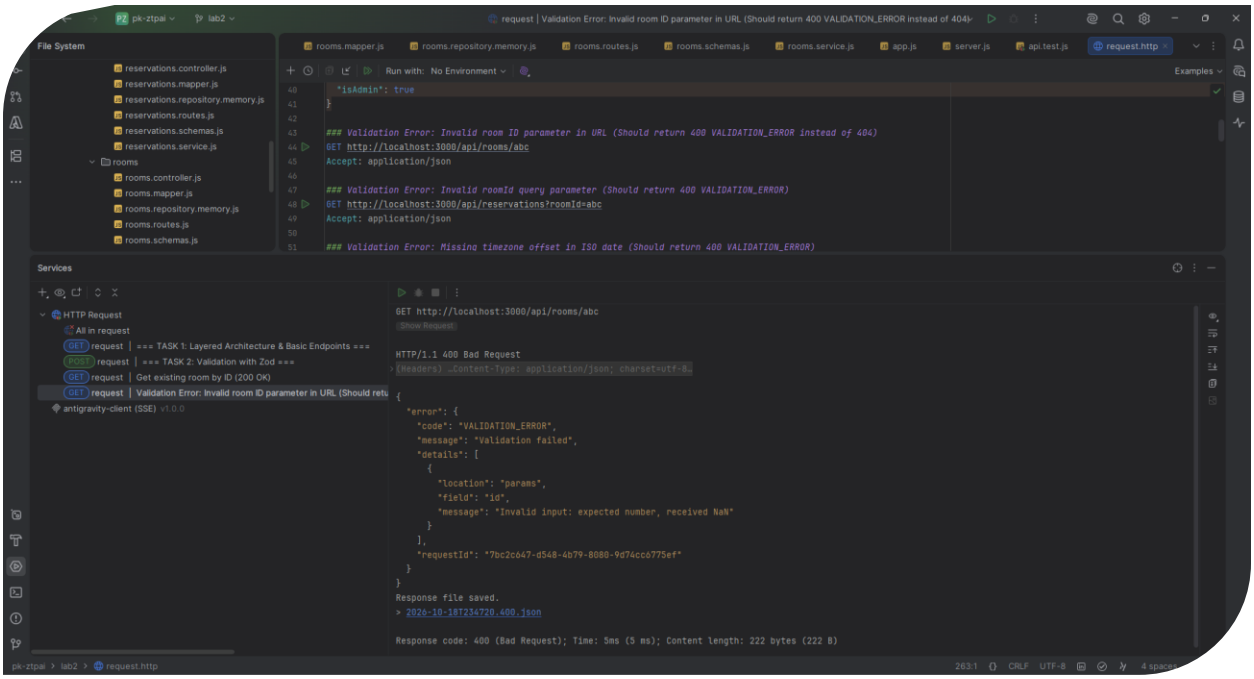
const idParams = z.object({
    id: z.coerce.number().int().positive()
});

const listReservationsQuery = z.object({
    roomId: z.coerce.number().int().positive().optional()
});

const createReservationBody = z.object({
    roomId: z.number().int().positive(),
    reservedBy: z.string().trim().min(3).max(100),
    participants: z.number().int().positive(),
    startTime: z.iso.datetime({ offset: true }),
    endTime: z.iso.datetime({ offset: true })
}).strict();
```



Zrzut ekranu z Ridera pokazujący request POST http://localhost:3000/api/reservations z wieloma błędami i odpowiedź 400 Bad Request zwracającą wszystkie błędy w tablicy details.



Zrzut ekranu z Ridera pokazujący walidację parametru ścieżki GET http://localhost:3000/api/rooms/abc zwracającą 400 Bad Request (zamiast 404).

Zadanie 3: Global error handling i logowanie

Wprowadzono hierarchię klas błędów dziedziczących po klasie bazowej `AppError` (`ValidationError, NotFoundError, ConflictError`). Dzięki wykorzystaniu natywnego wsparcia dla async handlerów w Express 5, kontrolery nie zawierają bloków try/catch ani bezpośrednich wywołań `res.status(4xx)`. Wszystkie błędy trafiają do centralnego middleware'a errorHandler, który formatuje odpowiedź do ujednoliconej struktury `{ error: { code, message, details?, requestId } }`. Dodano również requestLogger nadający żądaniom identyfikator `X-Request-Id` i mierzący czas wykonania.

Fragment kodu (hierarchia AppError, errorHandler oraz requestLogger):

```
// src/errors/AppError.js
class AppError extends Error {
    constructor(message, statusCode, code, details) {
        super(message);
        this.name = this.constructor.name;
        this.statusCode = statusCode;
        this.code = code;
        this.details = details;
    }
}

class ValidationError extends AppError {
    constructor(details, message = "Validation failed") {
        super(message, 400, "VALIDATION_ERROR", details);
    }
}
```

```
class NotFoundError extends AppError {
    constructor(resource) {
        super(`${resource} not found`, 404, "NOT_FOUND");
    }
}

class ConflictError extends AppError {
    constructor(message, code = "CONFLICT") {
        super(message, 409, code);
    }
}

// src/middleware/errorHandler.js
function errorHandler(err, req, res, next) {
    if (err instanceof AppError) {
        return res.status(err.statusCode).json({
            error: {
                code: err.code,
                message: err.message,
                details: err.details,
                requestId: req.id
            }
        });
    }

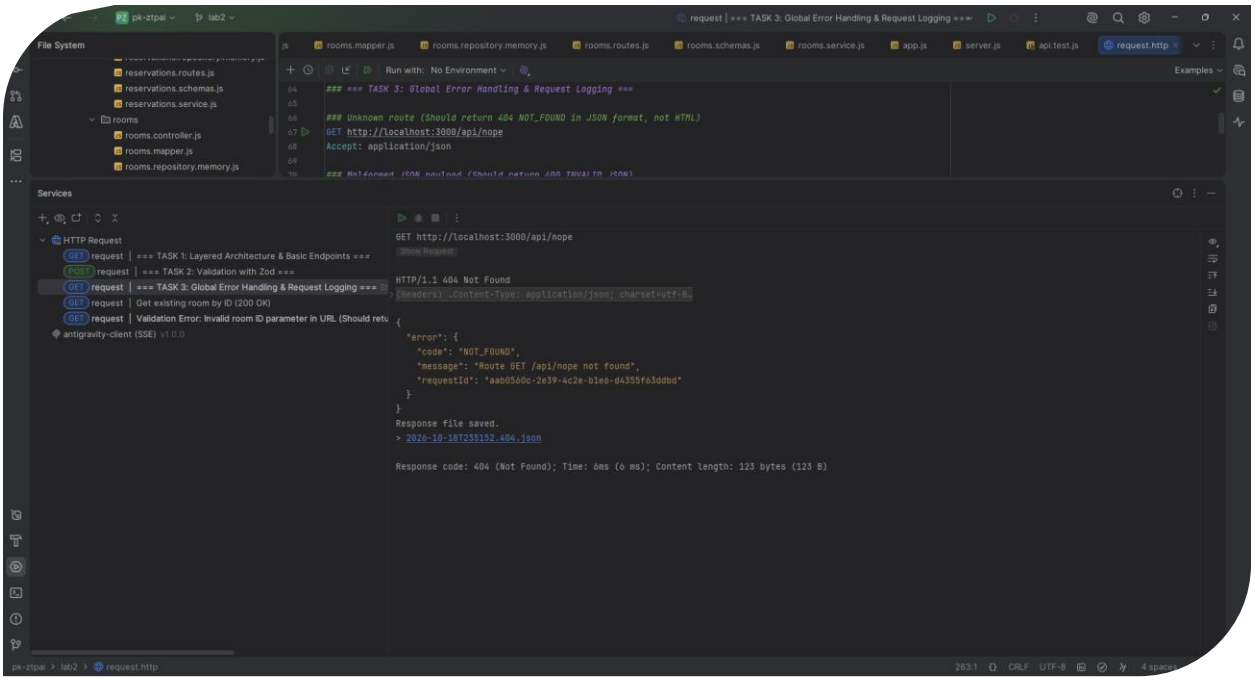
    if (err.type === "entity.parse.failed") {
        return res.status(400).json({
            error: {
                code: "INVALID_JSON",
                message: "Request body is not valid JSON",
                requestId: req.id
            }
        });
    }

    console.error(`[${req.id}]`, err);
    res.status(500).json({
        error: {
            code: "INTERNAL_ERROR",
            message: "Unexpected error",
            requestId: req.id
        }
    });
}

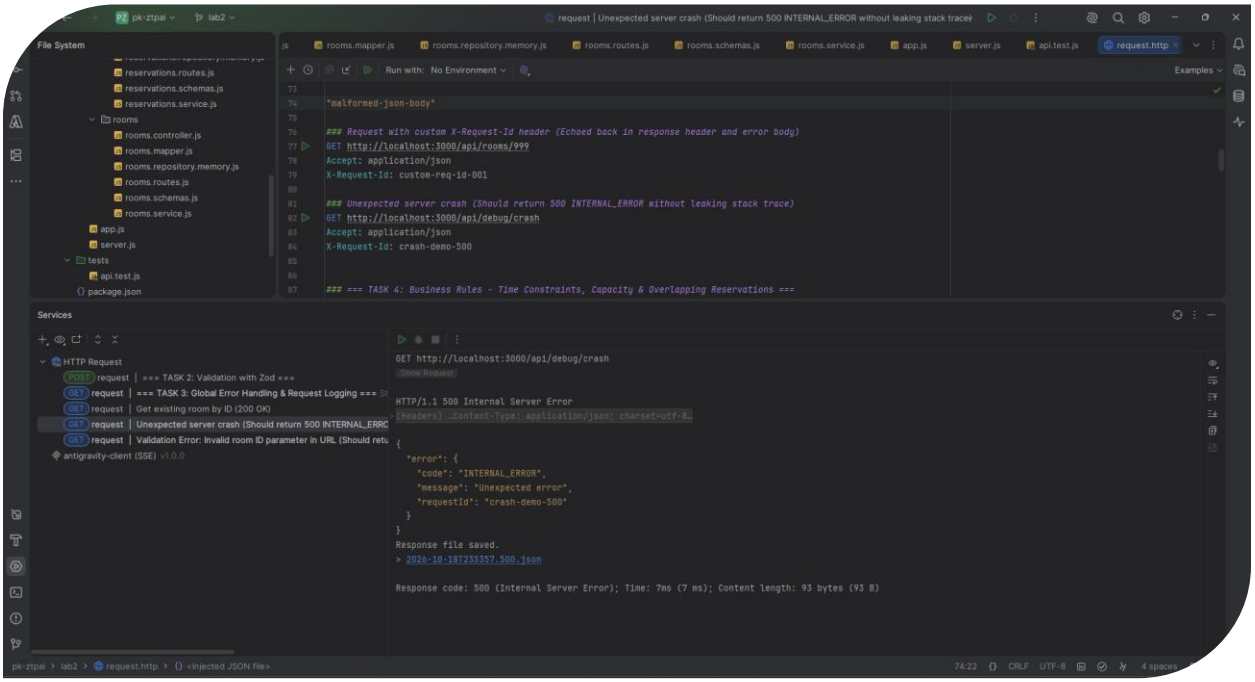
// src/middleware/requestLogger.js
function requestLogger(req, res, next) {
    req.id = req.get("X-Request-Id") || randomUUID();
    res.set("X-Request-Id", req.id);

    const start = process.hrtime.bigint();
    res.on("finish", () => {
        const ms = Number(process.hrtime.bigint() - start) / 1e6;
        console.log(`[${req.id}] ${req.method} ${req.originalUrl} -> ${res.statusCode}
(${ms.toFixed(1)} ms)`);
    });

    next();
}
```



Zrzut ekranu pokazujący obsługę nieistniejącej ścieżki GET http://localhost:3000/api/nope (404 NOT_FOUND w JSON) oraz uszkodzonego payloadu JSON (400 INVALID_JSON).



Zrzut ekranu pokazujący wywołanie diagnostycznego endpointu GET http://localhost:3000/api/debug/crash z nagłówkiem X-Request-Id: crash-demo-500.

Zadanie 4: Business rule – brak nakładających się rezerwacji

W warstwie `reservations.service.js` zaimplementowano reguły biznesowe operujące na wstrzykiwanym zegarze `now = () => new Date()` oraz znacznikach czasu `(getTime())`, co zapewnia poprawną obsługę różnych stref czasowych. System weryfikuje, czy `startTime` jest w przyszłości (`400`), czy `endTime > startTime` (`400`), czy czas trwania rezerwacji mieści się w przedziale od 15 minut do 8 godzin (`400`), czy liczba uczestników nie przekracza pojemności sali (`409 ROOM_CAPACITY_EXCEEDED`) oraz czy nowa rezerwacja nie nakłada się na inną aktywną (`CONFIRMED`) rezerwację w tej samej sali przy założeniu przedziałów półotwartych `[startTime, endTime) (409 ROOM_ALREADY_BOOKED)`.

Fragment kodu (reguły biznesowe i wykrywanie kolizji czasowych w reservations.service.js):

```
// src/modules/reservations/reservations.service.js (fragment createReservation)
const currentTime = now();
const startMs = new Date(input.startTime).getTime();
const endMs = new Date(input.endTime).getTime();
const nowMs = currentTime.getTime();

const validationDetails = [];

if (startMs <= nowMs) {
    validationDetails.push({
        location: "body",
        field: "startTime",
        message: "Start time cannot be in the past"
    });
}

if (endMs <= startMs) {
    validationDetails.push({
        location: "body",
        field: "endTime",
        message: "End time must be later than start time"
    });
} else {
    const durationMs = endMs - startMs;
    if (durationMs < MIN_DURATION_MS || durationMs > MAX_DURATION_MS) {
        validationDetails.push({
            location: "body",
            field: "endTime",
            message: "Reservation duration must be between 15 minutes and 8 hours"
        });
    }
}

if (validationDetails.length > 0) {
    throw new ValidationError(validationDetails);
}

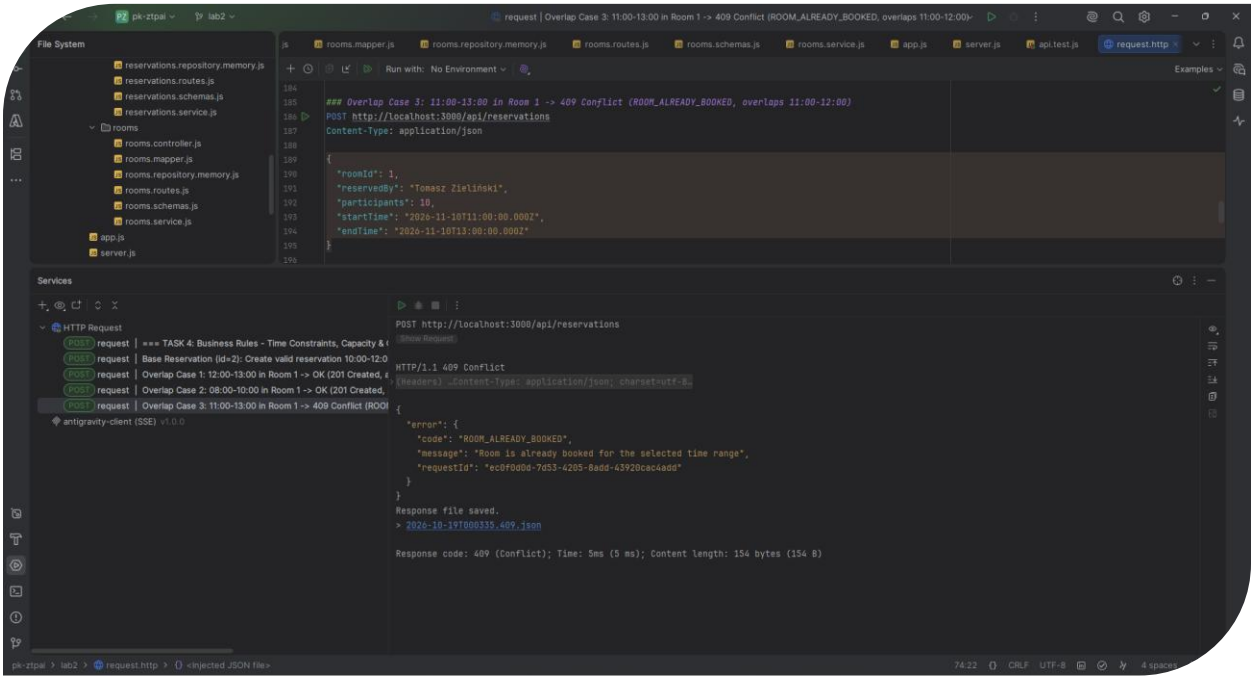
const room = await roomsRepository.findById(input.roomId);
if (!room) {
    throw new NotFoundError("Room");
}

if (input.participants > room.capacity) {
    throw new ConflictError("Room capacity exceeded", "ROOM_CAPACITY_EXCEEDED");
}
```

```
// Half-open intervals [startTime, endTime) overlap iff A.start < B.end AND B.start < A.end
const activeReservations = await reservationsRepository.list({
    roomId: input.roomId,
    status: "CONFIRMED"
});

const hasOverlap = activeReservations.some(existing => {
    const existingStartMs = new Date(existing.startTime).getTime();
    const existingEndMs = new Date(existing.endTime).getTime();
    return startMs < existingEndMs && existingStartMs < endMs;
});

if (hasOverlap) {
    throw new ConflictError(
        "Room is already booked for the selected time range",
        "ROOM_ALREADY_BOOKED"
    );
}
```



Zrzut ekranu prezentujący utworzenie rezerwacji bazowej 10:00–12:00 w sali nr 1 oraz 409 Conflict – ROOM_ALREADY_BOOKED

Zadanie 5: Anulowanie jako state machine + response DTO

Proces anulowania rezerwacji (`PATCH /api/reservations/:id/cancel`) został oparty o maszynę stanów zdefiniowaną jako struktura danych `ALLOWED_TRANSITIONS`. Anulowanie rezerwacji, która już się rozpoczęła (`startTime <= now`), jest blokowane błędem `409 Conflict` z kodem `RESERVATION_ALREADY_STARTED` (przetestowane na historycznej rezerwacji id: 1 z danych startowych repozytorium). Ponowne anulowanie rezerwacji o statusie `CANCELLED` jest idempotentne – zwraca kod `200 OK` oraz ten sam obiekt bez nadpisywania pola `cancelledAt`.

Wprowadzono również mappery DTO (`toRoomResponse, toReservationResponse`), które jawnie wskazują pola zwracane klientowi (`whitelist`), konwertują daty do standardu `ISO 8601 UTC (...Z)` i chronią przed wyciekiem wewnętrznego pola `internalNote: "created via API v2"`. Przy tworzeniu rezerwacji (`POST /api/reservations`) dodano nagłówek `Location: /api/reservations/{id}`.

Fragment kodu (maszyna stanów anulowania oraz mapper DTO):

```
// src/modules/reservations/reservations.service.js (state machine i cancelReservation)
const ALLOWED_TRANSITIONS = {
    CONFIRMED: ["CANCELLED"],
    CANCELLED: []
};

async function cancelReservation(id) {
    const reservation = await reservationsRepository.findById(id);
    if (!reservation) {
        throw new NotFoundError("Reservation");
    }

    const newStatus = "CANCELLED";

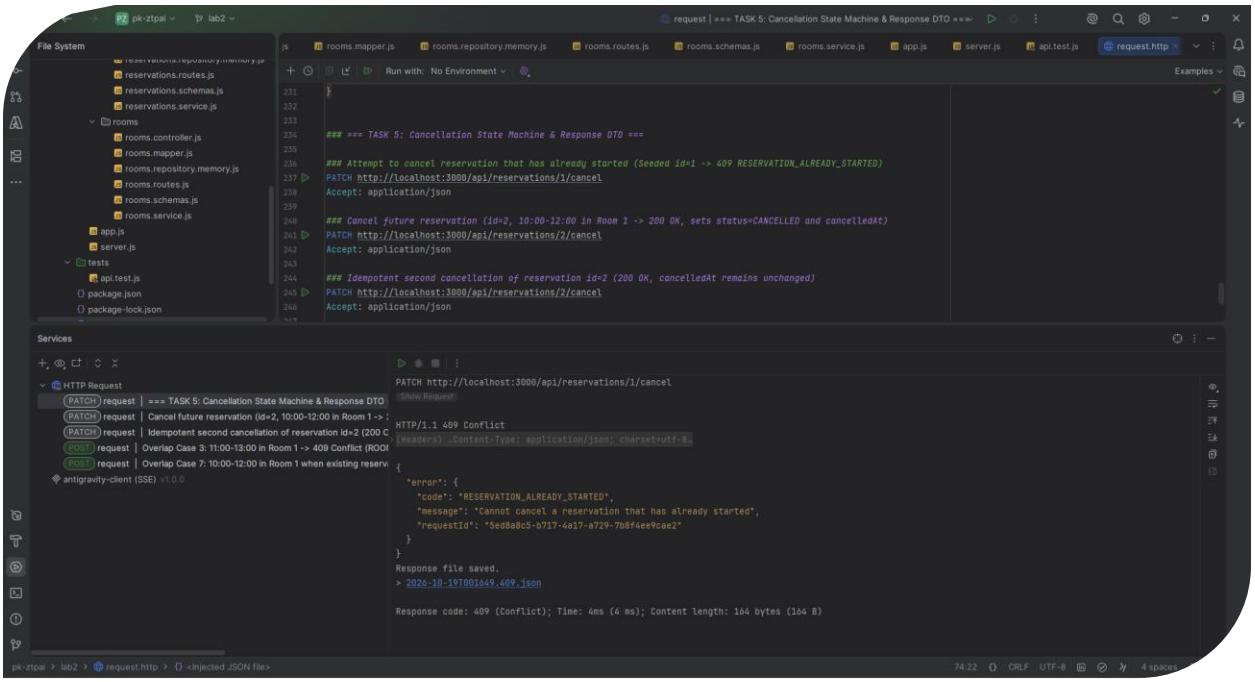
    // Idempotent operation: cancelling an already cancelled reservation returns it unchanged
    if (reservation.status === newStatus) {
        return reservation;
    }

    if (!ALLOWED_TRANSITIONS[reservation.status]?.includes(newStatus)) {
        throw new ConflictError(
            `Cannot change status from ${reservation.status} to ${newStatus}`,
            "INVALID_STATUS_TRANSITION"
        );
    }

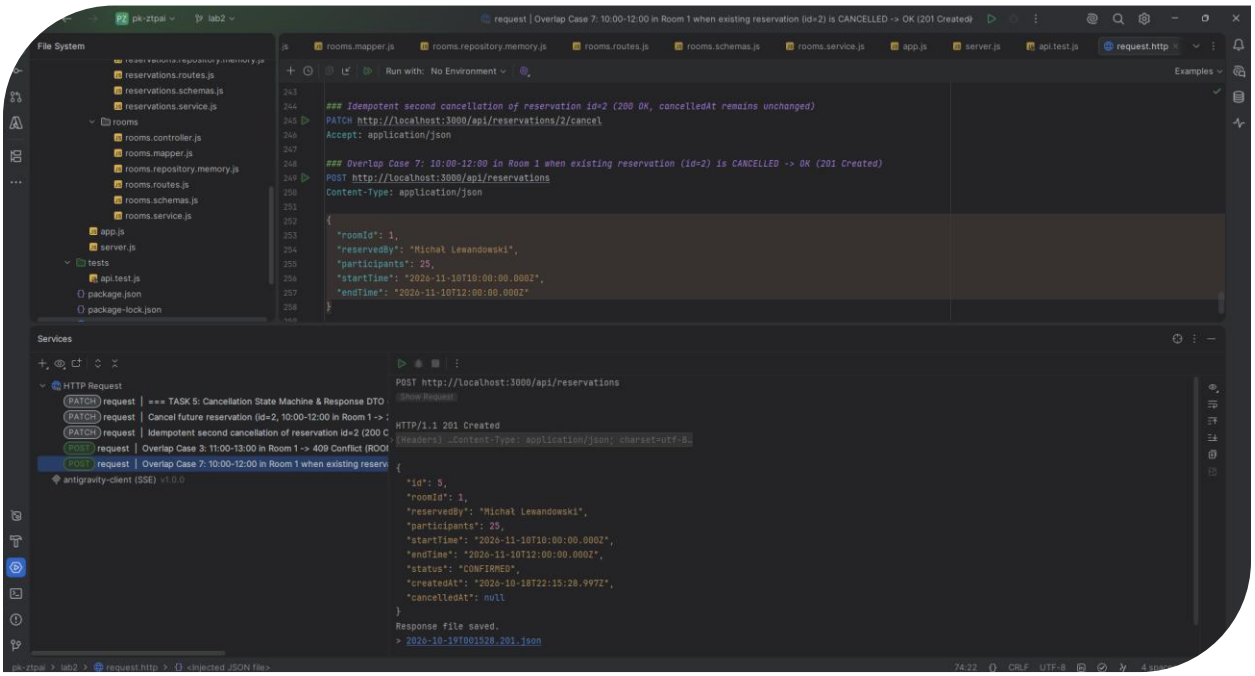
    const currentTime = now();
    if (new Date(reservation.startTime).getTime() <= currentTime.getTime()) {
        throw new ConflictError(
            "Cannot cancel a reservation that has already started",
            "RESERVATION_ALREADY_STARTED"
        );
    }

    return reservationsRepository.updateStatus(id, newStatus, {
        cancelledAt: currentTime.toISOString()
    });
}

// src/modules/reservations/reservations.mapper.js
function toReservationResponse(reservation) {
    return {
        id: reservation.id,
        roomId: reservation.roomId,
        reservedBy: reservation.reservedBy,
        participants: reservation.participants,
        startTime: new Date(reservation.startTime).toISOString(),
        endTime: new Date(reservation.endTime).toISOString(),
        status: reservation.status,
        createdAt: new Date(reservation.createdAt).toISOString(),
        cancelledAt: reservation.cancelledAt
            ? new Date(reservation.cancelledAt).toISOString()
            : null
    };
}
```



Zrzut ekranu pokazujący próbę anulowania rozpoczętej już rezerwacji ze zbioru startowego: PATCH http://localhost:3000/api/reservations/1/cancel zwracającą 409 Conflict z kodem RESERVATION_ALREADY_STARTED.



Zrzut ekranu pokazujący anulowanie przyszłej rezerwacji PATCH http://localhost:3000/api/reservations/2/cancel (200 OK, ustawienie status: "CANCELLED" oraz cancelledAt) oraz drugie wywołanie tego samego żądania zwracające 200 OK z identyczną, niezmienioną wartością cancelledAt (idempotentność).